LEMVOS ROBOTICS GMBH

LM450 Autonomous Navigation

Internship Technical Report

Item	Details
Prepared by	Rishabh Rudani
University	THWS
Program	Bachelor of Engineering in Robotics
Semester	5
Internship period	1 October 2025 - Present
University supervisor	Dorit Borman
Company supervisors	Simone Contorno and Phillip
Team	Robotics Software Development for USV
Document type	Main Report
Language	English

Abstract

This report presents the work I carried out on the LM450 autonomous navigation workspace during my internship at Lemvos Robotics GmbH. I worked on a ROS 2 Humble based marine-autonomy system for the LM450 unmanned surface vessel, with the goal of integrating localization, offline map generation, path planning, control, mission execution, and debugging into one practical workflow. The codebase is organized into five main authored packages: automatepro_localization, lm450_global_costmap, lm450_nav2, lm450_waypoints, and debugging_lm450_path. Together, they form a complete navigation stack for both simulation and real deployment.

1. Internship Context and Objective

My objective during the internship was not to work on one isolated node, but to contribute to a full navigation pipeline for a USV. The system is designed to move the LM450 autonomously through water by combining GNSS for global position, IMU and magnetometer data for heading and motion estimation, pre-generated water costmaps for traversability, Nav2 for planning and control, and BehaviorTree.CPP for task-level mission logic. Working on this stack gave me experience with estimation, mapping, control, mission behavior, and post-run analysis inside one coordinated robotics project.

2. Project Scope and Report Structure

2.1 Main Authored Packages

Package	Version	Primary role
automatepro_localization	2.0.0	GNSS / IMU fusion and EKF integration
lm450_global_costmap	1.1.0	Offline water costmap generation from geographic data
lm450_nav2	1.0.0	Nav2 planning, control, launch, and command remapping
lm450_waypoints	1.1.0	Behavior-tree mission execution and service interfaces
debugging_lm450_path	0.0.0	Path-following evaluation and logging

2.2 Supporting Workspace Content

- README.md as the main system guide
- TROUBLESHOOTING.md as an operator and debugging reference
- docs/PROMPT-REFACTORING.md as an internal engineering artifact
- Generated map outputs under lm450_global_costmap/output/
- A bundled Groot / BehaviorTree.CPP tooling tree

2.3 Boundary of this Report

In this report, I describe the authored LM450 packages in detail, include selected implementation excerpts, summarize the runtime interfaces, and explain the role of the bundled Groot tooling at a high level. I focus on the application logic that matters for the LM450 and on the parts of the workspace that were directly relevant to my internship work.

3. Workspace at a Glance

3.1 End-to-End Architecture

I organized the system as a layered autonomy stack so that each part had a clear responsibility and a clean interface to the next layer.

Layer	Main nodes / tools	Purpose
Sensors	GNSS, IMU, magnetometer	Provide global position, motion, and heading-related data
Localization	gps_to_pose, imu_mag_fuser, gps_velocity_estimator, ekf_node	Convert raw sensor data into a consistent fused vehicle pose
Mapping	generate_water_costmap.py, water_costmap.pgm/yaml	Prepare a water-based occupancy map for navigation
Navigation	map_server, planner_server, controller_server, bt_navigator, cmd_vel_remap	Plan paths and generate usable motion commands for the vessel
Mission	bt_runner, FollowMissionBT, UndockReverse, HoldPositionBT	Manage waypoint execution, undocking, and post-mission hold behavior
Debugging	path_follow_debugger, split_mcap_metrics.py	Measure and evaluate tracking performance offline and in RViz

3.2 Frame Ownership

Transform	Owner	Meaning
map -> odom	gps_to_pose	Anchors local odometry to the configured world frame
odom -> base_link	ekf_node	Publishes the fused vehicle pose used by the planner and controller
base_link -> sensor frames	Robot model / external TF sources	Represents sensor mounting geometry

3.3 Main Runtime Interfaces

Interface	Type	Purpose
/localization/gps_pose	PoseWithCovarianceStamped	GNSS converted into a local metric pose
/localization/imu/data_corrected	sensor_msgs/Imu	Corrected IMU orientation after magnetic fusion
/localization/gps_velocity	TwistWithCovarianceStamped	Planar velocity estimate derived from GPS pose updates
/localization/odometry/filtered	nav_msgs/Odometry	Final EKF output used by the rest of the stack
/follow_waypoints	Nav2 action	Waypoint mission execution
/compute_path_to_pose	Nav2 action	Hold-position replanning
/follow_path	Nav2 action	Correction execution
/cmd_vel	geometry_msgs/Twist	Raw navigation velocity command
/cmd_vel_nav_stamped	geometry_msgs/TwistStamped	Normalized output to the actuation layer
/mission/undock_active	std_msgs/Bool	Gate that allows reverse motion during undocking
/mission/new_goal	std_msgs/Bool	Exit signal for hold mode

4. Package Study: automatepro_localization

4.1 Purpose

In automatepro_localization, I built the state-estimation layer that converts raw GNSS, IMU, and magnetometer data into a fused pose that the rest of the navigation stack can trust. This package is responsible for giving the planner, controller, and mission logic a stable reference frame.

4.2 Main Components

File	Role
src/gps_to_pose.cpp	Converts latitude and longitude into a local metric pose and publishes map -> odom
src/imu_mag_fuser.cpp	Fuses gyroscope and magnetometer information to estimate yaw
src/gps_velocity_estimator.cpp	Computes planar velocity from GPS pose changes
launch/localization.launch.py	Starts the package together with robot_localization EKF
config/config_sim.yaml	Simulation parameters
config/config_lm450.yaml	Hardware parameters

4.3 Core Logic

- I convert every valid GNSS fix to UTM coordinates so the navigation stack works in metric space.
- The first valid fix defines the local odometry datum.
- I compute a fixed map -> odom translation from the configured map origin.
- IMU and magnetometer data are fused with a lightweight complementary filter for heading estimation.
- GPS-based velocity can be produced when needed for EKF fusion.
- robot_localization then fuses pose, heading, and velocity into /localization/odometry/filtered.

4.4 Key Implementation: GNSS Datum and Frame Anchoring

One of the most important ideas in this package is that the first valid GPS fix defines the local origin, while the map frame remains tied to the configured world origin. This keeps frame semantics consistent for every downstream planner and controller.

```
if (!datum_set_) {
  origin_easting_ = easting;
  origin_northing_ = northing;
  datum_set_ = true;
  map_to_odom_x_ = origin_easting_ - map_origin_easting_;
  map_to_odom_y_ = origin_northing_ - map_origin_northing_;
  map_to_odom_set_ = true;
}
const double odom_x = easting - origin_easting_;
const double odom_y = northing - origin_northing_;
```

4.5 Key Implementation: Lightweight Heading Fusion

I used a complementary-filter approach because it is simple, fast, and practical for embedded robotics. Gyroscope motion gives short-term responsiveness, while the magnetometer provides long-term yaw correction.

```
const float yaw_gyro = yaw_est_ + gyro_z * dt;
float new_yaw = yaw_est_;
if (use_mag) {
  const float diff = normalize_angle(yaw_mag - yaw_gyro);
  new_yaw = normalize_angle(yaw_gyro + (1.0f - alpha_) * diff);
} else {
```

```
  new_yaw = normalize_angle(yaw_gyro);
}
yaw_est_ = new_yaw;
```

4.6 Simulation versus Hardware

Parameter	Simulation	Hardware
base_link_frame	lm450/lm450/base_link	lm450_bow_link
EKF rate	5.0 Hz	20.0 Hz
GPS topic	simulated sensor topic	/sensor/gnss/position/fix
IMU topic	simulated IMU topic	physical IMU topic

5. Package Study: lm450_global_costmap

5.1 Purpose

In lm450_global_costmap, I prepared the offline maps that tell Nav2 where the vessel can move. Because this is a marine platform, the mapping logic is intentionally water-centric: water is traversable space and land remains non-navigable or unknown.

5.2 Data Sources and Main Files

File	Role
scripts/generate_water_costmap.py	Main generator
config/baggersee.yaml	Real-world operating area
config/vrx.yaml	Simulation area
launch/lm450_global_costmap.launch.py	Standalone map bring-up
requirements.txt	Python geospatial dependencies

5.3 Processing Pipeline

- Load water polygons from shapefiles and OpenStreetMap / OSMnx.
- Merge and dissolve the geometries into a usable water mask.
- Rasterize the water region.
- Encode water as free space, land as unknown, and the border as occupied.
- Convert metadata into UTM so Nav2 can plan in metric coordinates.
- Export PGM, YAML, and preview artifacts for repeated offline use.

5.4 Key Implementation: Marine-Centric Occupancy Encoding

The most important design decision in this package is the occupancy encoding itself. For a boat, water is free space, not an obstacle. I kept that idea explicit in the code and in the exported map values.

```
# 0 = free space (water)
# 100 = occupied (border)
# 255 = unknown (land)
free_val = config['occupancy_grid']['free_value']
unknown_val = config['occupancy_grid']['unknown_value']
occupied_val = config['occupancy_grid']['occupied_value']
occupancy_grid = np.where(water_raster == 1, free_val, unknown_val)
border_mask = (dilated_water == 1) & (water_mask == 0)
occupancy_grid[border_mask] = occupied_val
```

5.5 Generated Outputs

- water_costmap.pgm
- water_costmap.yaml
- water_all.gpkg
- water_mask_preview.png
- occupancy_grid_viz.png
- Example output sets such as baggersee_1m, baggersee_5m, baggersee_lake, vrx_1m, and vrx_5m

These outputs show that the package is designed for repeated offline map generation rather than continuous online mapping.

6. Package Study: lm450_nav2

6.1 Purpose

lm450_nav2 is the runtime navigation layer that I used to adapt Nav2 to the LM450 marine platform. This is where the system stops being generic and becomes specific to a real unmanned surface vessel.

6.2 Main Files and Functions

File	Role
launch/lm450_nav2.launch.py	Main navigation bring-up
config/config_sim.yaml	Simulation tuning
config/config_lm450.yaml	Hardware tuning
src/cmd_vel_remap.cpp	Velocity normalization, clamping, and marine logic
src/move_base_goal_bridge.cpp	Legacy goal bridging into Nav2 actions
src/smac_planner_expansions.cpp	Planner debug visualization
scripts/controller_config.py	Controller-limit calculation helper

6.3 Controller Philosophy

- Forward-oriented motion rather than in-place turning.
- Rotate-to-heading is disabled because it is often unsuitable for a boat.
- Lookahead behavior is tuned differently for simulation and for the real vessel.
- A reverse-enabled secondary controller is available for hold-position corrections.
- cmd_vel values are remapped from physical units into normalized actuation percentages.

6.4 Key Implementation: Behind-Point Recovery Behavior

A standard path-following controller does not handle every marine situation well, especially when the lookahead point falls behind the boat. I handled that case explicitly so the vessel can turn back toward the path while still respecting forward-biased marine behavior.

```
const bool point_is_behind = behind_point_control_enabled_ &&
                             std::isfinite(x_L) &&
                             x_L < 0.0;
if (point_is_behind) {
  const double heading_error = std::atan2(y_L, x_L);
  double turn_direction;
  if (std::abs(heading_error) < behind_point_heading_deadband_) {
    turn_direction = (last_turn_direction_ != 0.0) ? last_turn_direction_
                                                   : ((y_L >= 0.0) ? 1.0 : -1.0);
  } else {
    turn_direction = std::copysign(1.0, heading_error);
    last_turn_direction_ = turn_direction;
```

```
  }
  w_used = turn_direction * max_angular_velocity_;
  v_used = behind_point_linear_vel_;
}
```

6.5 Command Safety Layer

The cmd_vel_remap node also acts as a safety and integration layer. It clamps velocities, limits acceleration and deceleration, blocks reverse motion unless undocking is active, enforces a minimum effective command, and finally converts the result into normalized values in the range [-1.0, 1.0].

6.6 Simulation versus Hardware Tuning

Parameter	Simulation	Hardware	Meaning
desired_linear_vel	0.6	2.0	The real boat is tuned for a higher cruise speed
min_effective_linear_velocity	0.2	0.8	Hardware propulsion has a larger dead zone
lookahead behavior	velocity-scaled	fixed long lookahead	Hardware uses more anticipatory steering
frame_id	namespaced sim base link	lm450_bow_link	Platform-specific TF integration

6.7 Supporting Documentation

- docs/CUSTOM_CONTROL.md
- docs/CONTROLLER_CONFIG.md
- docs/RPPC.md

Keeping the controller rationale in documentation was important to me because tuning decisions are much easier to maintain when they are written down clearly.

7. Package Study: lm450_waypoints

7.1 Purpose

lm450_waypoints is the mission-execution layer and the most application-specific part of the project. This is where navigation becomes real vessel behavior.

7.2 What the Package Manages

- Undocking
- Handover to normal navigation
- Waypoint mission execution
- Post-mission hold-position correction
- Service-level triggering of behavior-tree subroutines

7.3 Main Files

File	Role
src/bt_runner.cpp	Lifecycle behavior-tree runner
src/follow_mission_bt.cpp	Plugin library with mission nodes
src/bt_node_service_runner.cpp	Service interface for subtree execution
config/mission_bt.xml	Tree definition
config/mission_sim.yaml	Simulation mission configuration
config/mission_lm450.yaml	Hardware mission configuration
include/lm450_waypoints/gps_utils.hpp	Geographic coordinate utilities

test/test_gps_conversion.cpp	Unit tests

7.4 Mission Flow

At a high level, the mission sequence follows a simple and readable chain: UndockReverse -> StopThrusters -> FollowMissionBT -> HoldPositionBT. Using BehaviorTree.CPP helped me keep the logic modular and easier to debug.

7.5 Key Implementation: GPS Conversion with Cross-Zone Fallback

Waypoints can create subtle geographic problems, especially when UTM zones do not match. I handled that case explicitly so the system does not fail silently.

```
const bool zones_match = (zone == origin_zone && is_northern == origin_northern);
if (!zones_match) {
  if (allow_cross_zone) {
    toLocalENUGeodesic(lat, lon, origin_lat, origin_lon, x, y);
    return;
  } else {
    throw std::runtime_error("UTM zone mismatch detected");
  }
}
```

7.6 Blackboard Wiring

I also used a blackboard-based parameter export so that XML behavior trees can consume ROS parameters directly without recompiling the plugins.

```
blackboard_ = BT::Blackboard::create();
blackboard_->set("node", shared_from_this());
auto ros_node = std::make_shared<rclcpp::Node>("bt_ros_node", opts);
blackboard_->set("ros_node", ros_node);
exportAllParametersToBlackboard();
```

7.7 Real Undock and Hold Logic

The undocking node supports both simulation and hardware behavior. In real operation, it can stop based on GPS radius when a valid target is available and fall back to distance-based logic when GPS is unavailable or stale. After the waypoint mission ends, HoldPositionBT checks drift relative to the final waypoint and uses Nav2 actions to plan and execute correction paths until the vessel is stable or a new goal arrives.

```
if (target_valid_ && gpsIsFresh(start_lat, start_lon)) {
  x_distance_m_ = haversineMeters(start_lat, start_lon, target_lat_deg_, target_lon_deg_);
  real_mode_ = RealUndockMode::GPS_TO_RADIUS;
} else if (fallback_on_no_gps_ || !target_valid_) {
  real_mode_ = RealUndockMode::FALLBACK_DISTANCE;
} else {
  stopCmdVel();
}
```

For hold correction, I intentionally reused Nav2 actions such as ComputePathToPose and FollowPath instead of writing a separate low-level correction controller. That kept the design cleaner and more maintainable.

7.8 Service Interface and Tests

- Service endpoints include run_full_mission, run_undock, run_stop_thrusters, run_follow_mission, and run_hold_position.
- The service layer supports preemption so testing and debugging can interrupt a full mission cleanly.
- The GPS utility tests check UTM conversion, same-zone ENU conversion, geodesic fallback, strict mismatch behavior, hemisphere handling, and long-distance consistency.

8. Package Study: debugging_lm450_path

8.1 Purpose

I created debugging_lm450_path as a validation package so I could measure how well the vessel followed a planned path instead of relying only on visual inspection.

8.2 Metrics Produced

- Closest point on the path
- Signed cross-track error
- Along-track distance
- Reference yaw
- Heading error
- CSV logging
- Optional MCAP logging
- RViz marker output

8.3 Key Implementation: Closest-Point Projection

The core geometry projects the current odometry sample onto the nearest path segment and then derives signed error and reference heading from that projection.

```
double t = (apx * abx + apy * aby) / ab2;
t = std::clamp(t, 0.0, 1.0);
geometry_msgs::msg::Point proj;
proj.x = a.x + t * abx;
proj.y = a.y + t * aby;
const double cross = abx * apy - aby * apx;
const double sign = (cross >= 0.0) ? 1.0 : -1.0;
best.cte_signed = sign * dist;
best.along_s = cum_s_[i] + t * seg_len;
best.ref_yaw = std::atan2(aby, abx);
```

8.4 Supporting Script

The split_mcap_metrics.py script separates recorded CSV-row MCAP bags into individual metrics such as cte_m, nearest_distance_m, along_track_s_m, and heading_err_rad. That made replay, plotting, and controller analysis much easier.

9. Bundled Tooling: Groot and BehaviorTree.CPP

The workspace also includes a bundled Groot tree together with BehaviorTree.CPP-related tooling. I treat this as supporting developer infrastructure rather than as the main LM450 application code. Its value is that it supports tree authoring, visualization, and monitoring for the mission package.

- Groot GUI editor
- QtNodeEditor
- Vendored BehaviorTree.CPP source tree
- Resources, tests, and installer assets

10. Simulation and Hardware Comparison

10.1 Localization

Category	Simulation	Hardware
Base frame	lm450/lm450/base_link	lm450_bow_link
EKF frequency	5.0 Hz	20.0 Hz
Sensors	simulated topics	hardware topics

10.2 Nav2

Category	Simulation	Hardware
Cruise speed	0.6 m/s	2.0 m/s
Dead-zone floor	0.2 m/s	0.8 m/s
RViz default	enabled in sim configuration	disabled in hardware configuration

10.3 Mission Execution

Category	Simulation	Hardware
use_real_undock	false	true
use_sim_dock	true	false
loop_mission	false	true
Latch control	disabled	enabled

11. Engineering Practices Observed

11.1 Clear Separation of Concerns

- Localization is kept separate from planning.
- Planning is kept separate from mission orchestration.
- Offline mapping is separated from runtime navigation.
- Debugging lives in its own package instead of being mixed into production code.

11.2 Operational Fallbacks and Discipline

- GPS freshness checks are applied where they matter.
- Cross-zone geographic fallback avoids silent coordinate errors.
- Real undocking can fall back to distance-based logic.
- Hold-position correction uses replanning rather than brittle ad hoc behavior.
- Service-level preemption makes testing and debugging safer.

11.3 Embedded Pragmatism

- Preallocated messages appear where repeated allocations would be wasteful.

- Logging is throttled to avoid flooding the console.
- YAML-driven configuration keeps tuning practical.
- Simulation and hardware configurations are intentionally kept separate.

12. Conclusion

This internship gave me the opportunity to work on a complete marine-autonomy stack instead of a single isolated algorithm. By building and integrating localization, water-map generation, Nav2 planning and control, behavior-tree mission logic, and path-following analysis, I was able to understand how an autonomous USV becomes reliable in practice. The most important takeaway for me was that each package had to do its own job clearly, while all of them also had to fit together through stable frames, clean interfaces, and sensible fallback behavior. That combination is what makes the LM450 workspace practical for both simulation and real deployment.

Appendix A. Authored Package Inventory

Package	Summary
automatepro_localization	GNSS / IMU fusion and EKF integration
lm450_global_costmap	Offline water-map generation
lm450_nav2	Nav2 integration, tuning, and command remapping
lm450_waypoints	Behavior-tree mission execution and services
debugging_lm450_path	Path-following evaluation and logging